Take-Home Coding Exercise — Full-Stack Developer

Solution25 · Estimated time: 3–4 hours

Overview

Build a small Product Inventory & Order Management service. This is a deliberately scoped slice of an ecommerce backend with a thin frontend. We care more about how you build it than how much you build — clean architecture, sensible trade-offs, and working code beat feature count.

■■ **Timebox it to ~4 hours.** If you run out of time, stop and document what you'd do next. We explicitly reward a clear “what I’d improve” section over rushed, half-finished features.

The Task

A small shop needs to manage products and place orders against available stock.

Required functionality

Backend (API)

- CRUD for Products (*name*, *sku*, *price*, *stockQuantity*, *category*)
- Create an Order containing one or more products with quantities
- When an order is placed, it must:
  - Validate that requested quantities are in stock
  - Decrement stock atomically (no overselling under concurrent requests)
  - Calculate and persist the order total
- List orders with their line items and computed totals
- Basic input validation with meaningful error responses

Frontend (thin UI)

- A product list view (with stock levels)
- A simple “create order” flow (select products + quantities, submit)
- Display order confirmation with the total

Deliberate constraint

Treat the stock decrement as a concurrency problem. Two orders for the last item in stock should not both succeed. How you solve this (DB transactions, row locking, optimistic concurrency, etc.) is up to you — explain your choice.

Tech Requirements

- **Language:** TypeScript (frontend and backend)
- **Frontend:** Vue.js or React

• **Backend:** Node.js (any framework — Nest, Fastify, Express) or Laravel if you prefer PHP

• **Database:** Your choice (Postgres, MySQL, MongoDB) — justify relational vs. non-relational for this domain

• Must run via *docker compose up* from a clean checkout, or include clear setup steps if not Dockerized

What We're Evaluating

Since this round emphasizes full-stack breadth and architecture & best practices, we're specifically looking at:

Area	What we look for
Architecture	Clear separation of concerns (routing / business logic / data access). Not everything in one file.
Data modeling	Sensible schema, correct relationships, appropriate constraints/indexes.
Correctness under concurrency	The stock-decrement problem is handled, not ignored.
API design	RESTful (or documented alternative), consistent, proper status codes and error shapes.
Code quality	Readable, typed, consistent. Design patterns applied where they earn their place — not over-engineered.
Testing	At least a few meaningful tests around the order/stock logic. Breadth over exhaustiveness.
Git hygiene	Incremental, readable commits. Not one giant “final” commit.
Docs	A README we can follow without asking you questions.

Deliverables

1. A Git repository (link or zip) with full history.
2. A README containing:
   – Setup/run instructions
   – Your key architectural decisions and trade-offs
   – DB choice justification
   – How you handled concurrent stock updates
   – “What I’d do with more time” section
3. Tests covering the core order logic.

Stretch Goals (optional — only if time allows)

Pick at most one. We'd rather see the core done well.

- Containerize cleanly with multi-stage Docker builds + a working docker-compose.yml
- Add a simple CI workflow (lint + test on push)
- Add basic auth (e.g. an API key or JWT) protecting write endpoints
- Add pagination/filtering to the product list